EXPERIMENT - 2

NUMPY OPERATIONS

AIM:

To study and practice various NumPy operations including array creation, attributes, indexing, slicing, broadcasting, arithmetic, statistical operations, concatenation, reshaping, sorting and splitting using a case study.

Case Study:

A company manager wants to analyze bike sales over 4 weeks in January from their three branches (branch a, b and c). Using NumPy, extract meaningful insights from the raw sales data.

PREREQUISITES & REQUIREMENTS:

1. Computer with Python Installed
2. Jupytor Notebook
3. Knowledge on Python & Numpy Library

Step 1: Install and Import the NumPy Library using commands:

pip install numpy

In [71]:
```
# importing NumPy
import numpy as np
```

Step 2: Create arrays for three branches a, b and c for four week bike sales

In [72]:
```
# [Week1, Week2, Week3, Week4] for branch _241UDS0092_a,_241UDS0092_b and _241UDS0092_c
_241UDS0092_a = np.array([120, 100, 130, 300])
_241UDS0092_b = np.array([180, 110, 110, 200])
_241UDS0092_c = np.array([190, 260, 280, 290])
```

In [73]:
```
# Created arrays
_241UDS0092_a,_241UDS0092_b,_241UDS0092_c
```

Out[73]:
```
(array([120, 100, 130, 300]),
 array([180, 110, 110, 200]),
 array([190, 260, 280, 290]))
```

Step 3: Check the properties of the arrays with different attributes

In [74]:
```
# Dimension of the array
_241UDS0092_a.ndim
```

Out[74]: 1

In [75]:
```
# Shape of the array
_241UDS0092_b.shape
```

Out[75]: (4,)

In [21]:
```
# Size of the array
_241UDS0092_c.size
```

Out[21]: 4

In [76]:
```
# Data Type of the array
_241UDS0092_a.dtype
```

Out[76]: dtype('int64')

In [77]:
```
# Space used by each element in array
_241UDS0092_a.itemsize, _241UDS0092_b.itemsize, _241UDS0092_c.itemsize
```

Out[77]: (8, 8, 8)

Step 4: Retretive Sales data by Indexing & Slicing the arrays

In [78]:
```
# Created arrays
_241UDS0092_a, _241UDS0092_b, _241UDS0092_c
```

Out[78]:
```
(array([120, 100, 130, 300]),
 array([180, 110, 110, 200]),
 array([190, 260, 280, 290]))
```

In [79]:
```
# Index start with 0
# Get the _241UDS0092_c week 1 sales data
_241UDS0092_c[0]
```

Out[79]: np.int64(190)

In [82]:
```
# Get the unique values in the _241UDS0092_b
np.unique(_241UDS0092_b)
```

Out[82]: array([110, 180, 200])

In [83]:
```
# Get_241UDS0092_a week 1,2 sales data
_241UDS0092_a[0:2]
```

Out[83]: array([120, 100])

In [28]:
```
# Get _241UDS0092_b last two weeks sales data
_241UDS0092_b[2:]
```

Out[28]: array([110, 200])

In [84]:
```
# Get _241UDS0092_c last three weeks sales data (negative slicing)
_241UDS0092_c[-3:]
```

Out[84]: array([260, 280, 290])

Step 5: Assign new values to the branch sales (Broadcasting)

In [30]:
```
_241UDS0092_a
```

Out[30]:
```
array([120, 100, 130, 300])
```

In [31]:
```
# Making a new copy for_241UDS0092_a
_241UDS0092_a_new = _241UDS0092_a
```

In [33]:
```
_241UDS0092_a_new
```

Out[33]:
```
array([120, 100, 130, 300])
```

In [85]:
```
# Updating week 1 sales data
_241UDS0092_a_new[0] = 193
_241UDS0092_a_new
```

Out[85]:
```
array([193, 100, 130, 300, 234, 450])
```

In [86]:
```
# Adding week 5,6 sales data to the _241UDS0092_a_new array
_241UDS0092_a_new = np.append(_241UDS0092_a_new, [230,140])
_241UDS0092_a_new
```

Out[86]:
```
array([193, 100, 130, 300, 234, 450, 230, 140])
```

In [87]:
```
#Updating week 5, 6 sales data using slicing
_241UDS0092_a_new[4:6] = [234,450]
_241UDS0092_a_new
```

Out[87]:
```
array([193, 100, 130, 300, 234, 450, 230, 140])
```

Step 6: Arithmetic Operations on the sales data

In [39]:
```
_241UDS0092_c
```

Out[39]:
```
array([190, 260, 280, 290])
```

In [88]:
```
# Adding +2 sales to all 4 weeks
_241UDS0092_c+2
```

Out[88]:
```
array([192, 262, 282, 292])
```

In [89]:
```
# Subtracting 100 sales from the week 4 data with indexing in 241UDS0092_c
_241UDS0092_c-10
```

Out[89]:
```
array([180, 250, 270, 280])
```

In [42]:
```
# Double the sales in week 1 with indexing in 241UDS0092_c
_241UDS0092_c[0]*2
```

Out[42]:
```
np.int64(380)
```

In [90]:
```
# Half sales in week 2 & 3
_241UDS0092_c[1:3]/2
```

Out[90]:
```
array([130., 140.])
```

Step 7: Statistical Operations on the sales data

In [91]:
```
_241UDS0092_a, _241UDS0092_b, _241UDS0092_c
```

Out[91]:
```
(array([120, 100, 130, 300]),
 array([180, 110, 110, 200]),
 array([190, 260, 280, 290]))
```

In [92]:
```
# What are the Total sales of  _241UDS0092_a
np.sum(_241UDS0092_a)
```

Out[92]:
```
np.int64(650)
```

In [93]:
```
# What are the Maximum and Minumun sales of the _241UDS0092_a
np.max(_241UDS0092_a), np.min(_241UDS0092_a)
```

Out[93]:
```
(np.int64(300), np.int64(100))
```

In [94]:
```
# What are the average (mean) sales of the each branch
print(np.mean(_241UDS0092_a), np.mean(_241UDS0092_b), np.mean(_241UDS0092_c))
```

```
162.5 150.0 255.0
```

Step 8: Combine all branches sales data (Concatenating)

In [95]:
```
_241UDS0092_a, _241UDS0092_b, _241UDS0092_c
```

Out[95]:
```
(array([120, 100, 130, 300]),
 array([180, 110, 110, 200]),
 array([190, 260, 280, 290]))
```

In [96]:
```
_241UDS0092_a.ndim, _241UDS0092_b.ndim,_241UDS0092_c.ndim
```

Out[96]:
```
(1, 1, 1)
```

In [97]:
```
# Combining all 3 branches sales
all_branches_sales = np.concatenate([_241UDS0092_a, _241UDS0092_b, _241UDS0092_c])
```

In [98]:
```
all_branches_sales
```

Out[98]:
```
array([120, 100, 130, 300, 180, 110, 110, 200, 190, 260, 280, 290])
```

In [99]:
```
# Total sales of all branches
np.sum(all_branches_sales)
```

Out[99]:
```
np.int64(2270)
```

Step 9: Reshape the all_branches_sales into 3x4 matrix

In [100…
```
all_branches_sales.ndim, all_branches_sales.shape
```

Out[100…
```
(1, (12,))
```

In [101…
```
all_branches_sales = all_branches_sales.reshape(3,4)
all_branches_sales
```

Out[101…
```
array([[120, 100, 130, 300],
       [180, 110, 110, 200],
       [190, 260, 280, 290]])
```

In [102…
```
# Check the shape and dimension
all_branches_sales.ndim, all_branches_sales.shape
```

Out[102…
```
(2, (3, 4))
```

Step 10: Sort and Split the all_branches_sales array

In [103…
```
all_branches_sales
```

Out[103…
```
array([[120, 100, 130, 300],
       [180, 110, 110, 200],
       [190, 260, 280, 290]])
```

In [104…
```
# Indexing 2-D array
all_branches_sales[0]
```

Out[104…
```
array([120, 100, 130, 300])
```

In [105…
```
# Sort Row wise sales
np.sort(all_branches_sales, axis=1)
```

Out[105…
```
array([[100, 120, 130, 300],
       [110, 110, 180, 200],
       [190, 260, 280, 290]])
```

In [106…
```
# Sort Column wise sales
np.sort(all_branches_sales, axis=0)
```

Out[106…
```
array([[120, 100, 110, 200],
       [180, 110, 130, 290],
       [190, 260, 280, 300]])
```

In [107…
```
# Splitting the all_branches_sales array into 3 different arrays
# Splitting based on rows (axis=0)
split_92a, split_92b, split_92c = np.split(all_branches_sales, [1,2], axis=0)
```

In [108…
```
split_92a
```

Out[108…
```
array([[120, 100, 130, 300]])
```

In [109…
```
split_92b
```

Out[109…
```
array([[180, 110, 110, 200]])
```

In [110…
```
split_92c
```

Out[110…
```
array([[190, 260, 280, 290]])
```

In [111…
```
all_branches_sales
```

Out[111…
```
array([[120, 100, 130, 300],
       [180, 110, 110, 200],
       [190, 260, 280, 290]])
```

In [112…
```
# What are the Week 2 sales of all three branches? # Slice only column 2
week_2_sales = all_branches_sales[:3, 1:2]
```

In [113…
```
print(week_2_sales)
```

```
[[100]
 [110]
 [260]]
```

RESULT:

By using the NumPy library, various operations on arrays such as creation, attribute access, indexing, slicing, broadcasting, arithmetic operations, statistical computations, concatenation, sorting, and splitting were successfully implemented and demonstrated using a bike sales data.

In [ ]: